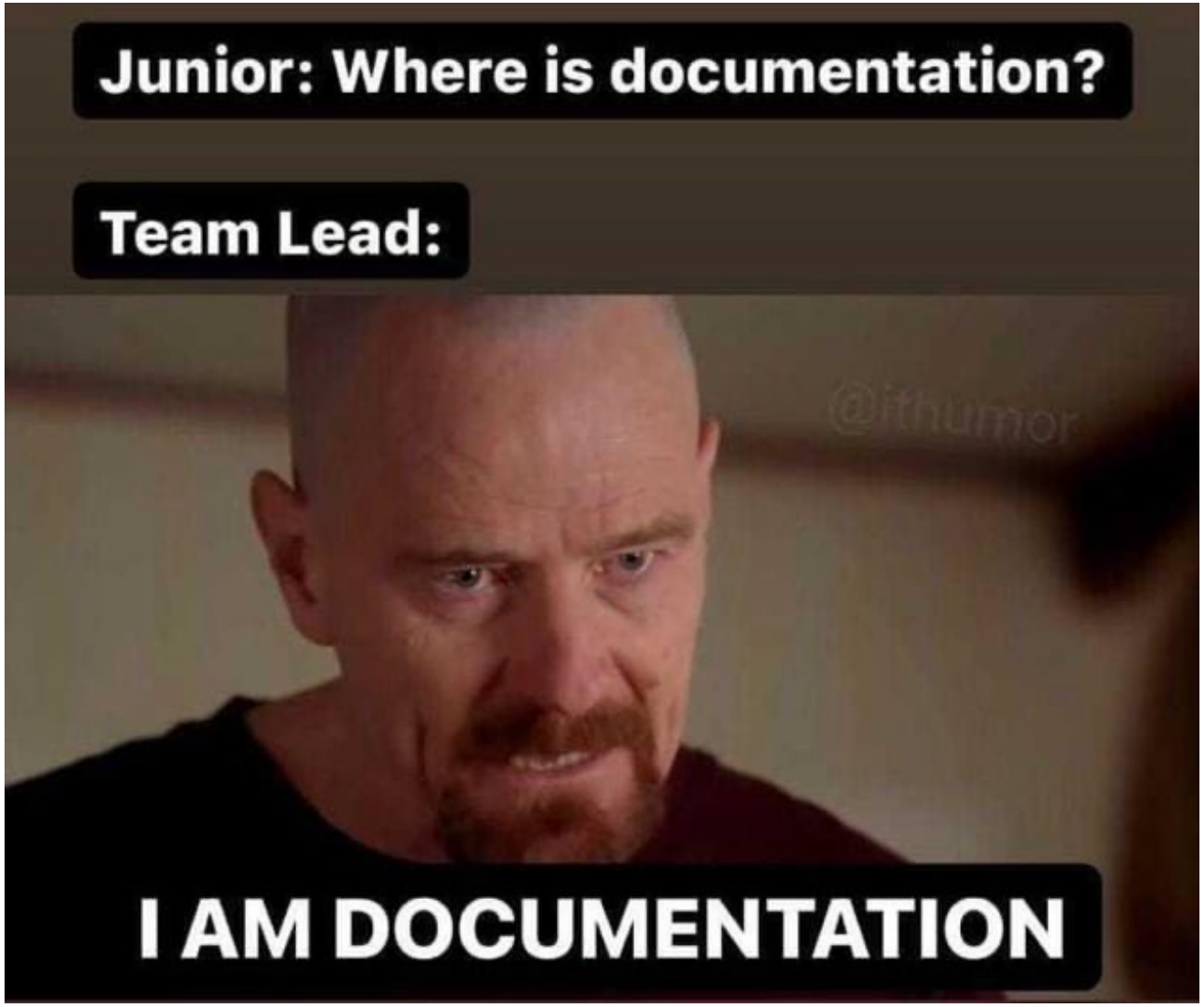
Introduction to the Software Design Document (SDD)



What Is a Software Design Document?

- A structured blueprint that describes how a system will be built.
- Bridges requirements and implementation.
- Ensures consistency, clarity, and alignment across stakeholders.
- ✓ Used by: developers, architects, QA, managers
- 📁 Output: diagrams, design decisions, module specs

Why Write an SDD?

- Clarifies architecture before coding begins
- Enables team collaboration and future onboarding
- Useful for risk analysis and trade-off discussion
- Acts as reference during testing and maintenance
- 💬 “Code is not self-explanatory at the system level.”

What Goes Into an SDD?

- 1. Overview – purpose, scope, glossary
- 2. Architecture – high-level design, major components
- 3. Data Design – schemas, storage, relationships
- 4. Component Design – interfaces, responsibilities
- 5. Interaction Design – UML diagrams, flows
- 6. Non-Functional Requirements – scalability, security
- 7. Technology Stack – tools, libraries, frameworks
- 8. Appendix – references, change log

- 🧠 Not all sections are mandatory; adapt per project size.

Common Diagrams in an SDD

- UML Diagrams: Class / Sequence / Activity / State
- Architecture views: Component, Deployment
- Data Flow Diagrams (DFD)
- ERDs (Entity Relationship Diagrams)
- API Interface Contracts
-  Visuals clarify complexity and reduce ambiguity.

SDD Stakeholders

- Architects: define system design
- Developers: reference during implementation
- QA/Testers: use for test planning
- Managers: assess scope, risks, milestones
- New team members: onboarding and understanding rationale
- ✍️ It’s a living document – evolve it with the code.

SDD Mistakes to Avoid

- Too high-level: lacks actionable details
- Too low-level: becomes a code dump
- Outdated content: diverges from reality
- Not maintained: loses value over time
- Created as formality: not actually used
- 🧠 Make it useful, not just beautiful.

Writing a Good SDD

- Keep it clear, consistent, concise
- Use version control (e.g., Git or Confluence)
- Include diagrams and rationale for decisions
- Reference requirements and link to source code
- Review regularly with the team
- 💡 “Design is communication. The SDD is its language.”